Lesson: Introduction to Authentication & Authorization

Introduction

Access control is a fundamental aspect of securing modern systems and protecting sensitive data. At its core, access control involves two key processes: authentication and authorization. Authentication verifies the identity of users or entities attempting to access a system, while authorization determines what resources or actions they are permitted to access. This lesson explores the foundations of access control, including various authentication methods, authorization models, and practical implementations like token-based authentication, Passport.js, and Role-Based Access Control (RBAC). By understanding these concepts, you will gain the knowledge needed to design secure and efficient systems.

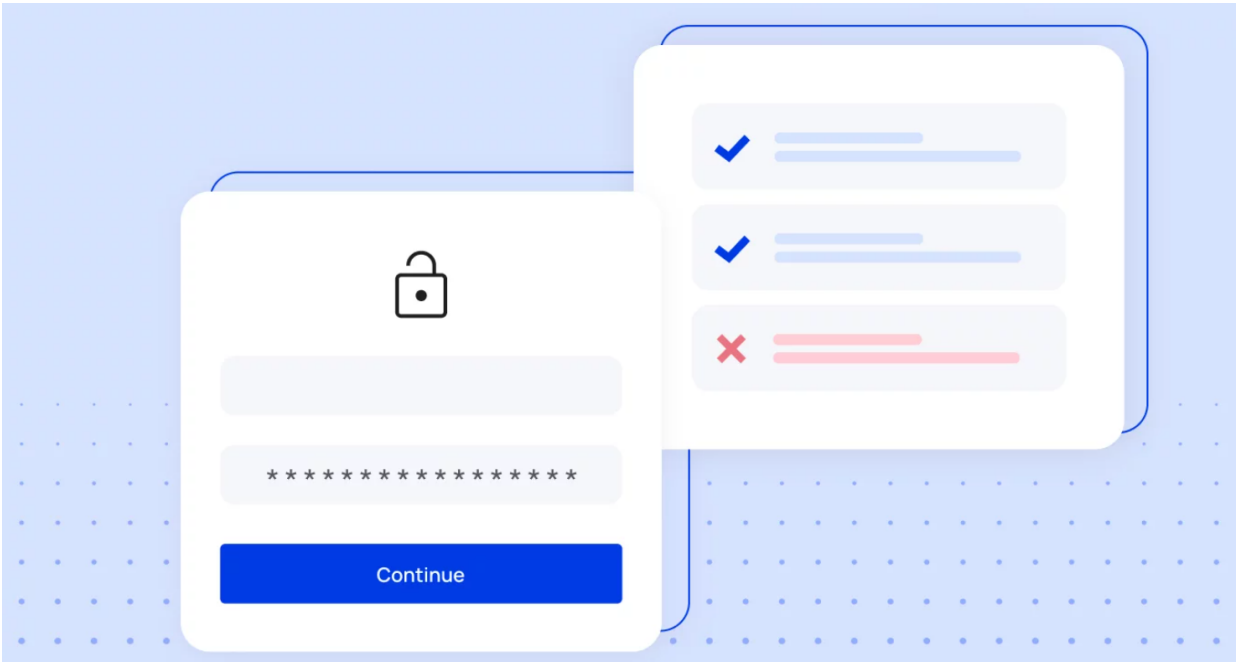
Learning Outcomes

By the end of this lesson, you will be able to:

1. Differentiate between authentication and authorization and understand their roles in access control.
2. Identify and explain various authentication methods, such as passwords, biometrics, and Multi-Factor Authentication (MFA).
3. Describe common authorization models, including Discretionary Access Control (DAC), Role-Based Access Control (RBAC), and Attribute-Based Access Control (ABAC).
4. Implement session-based and token-based authentication strategies, including JSON Web Tokens (JWT).
5. Use tools like Passport.js to integrate authentication into web applications.
6. Apply Role-Based Access Control (RBAC) to protect API endpoints and enforce permissions based on user roles.

Understanding the Foundations of Access Control

Authentication and authorization are two critical components of access control, often used together to ensure that only legitimate users can access protected resources. While they are closely related, these terms describe distinct processes. Below, we will explore what authentication and authorization mean, their importance, and the various methods used to implement them.



Authentication: Verifying Identity

Authentication is the process of verifying the identity of a user or entity attempting to access a system. It ensures that the person or device requesting access is who or what they claim to be. This verification is typically done by comparing provided credentials (such as usernames and passwords) against an authorized database. Authentication plays a vital role in securing systems, protecting sensitive information, and ensuring that only trusted individuals can access specific resources. For example, when you log into your email account, the system checks whether the password you entered matches the one stored in its database. If it does, the system grants access, confirming your identity.

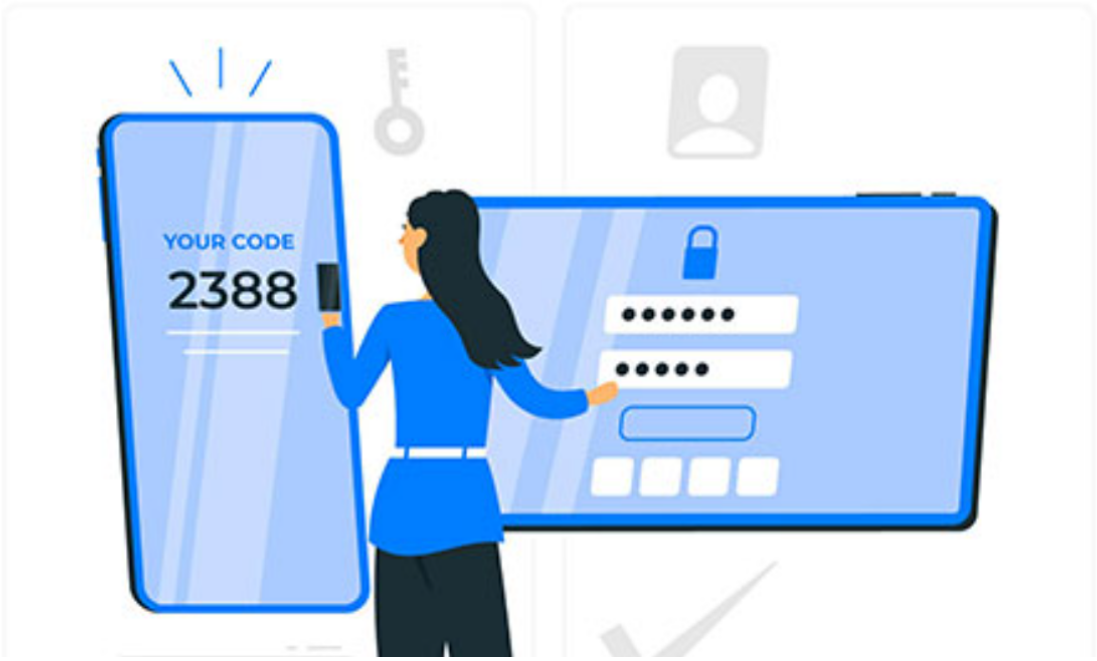
Authentication applies at various levels of a network, including ports, hosts, and services. By limiting access to authorized users, organizations can safeguard their data and infrastructure from unauthorized use or malicious attacks.

Types of Authentication Methods

There are several ways to authenticate users, each with varying levels of security and complexity. Below are some of the most common methods:

1. **Passwords**:
   Passwords remain one of the most widely used forms of authentication, despite being considered less secure compared to modern alternatives. They require users to create and remember a combination of characters to prove their identity. However, weak or reused passwords can make systems vulnerable to attacks.

2. **One-Time Passwords (OTP)**:
   OTPs provide a temporary code that grants access for a single session or transaction. These codes are typically sent via email, text message, or generated by a physical token. Because they expire after use, OTPs add an extra layer of security, making it harder for attackers to exploit stolen credentials.

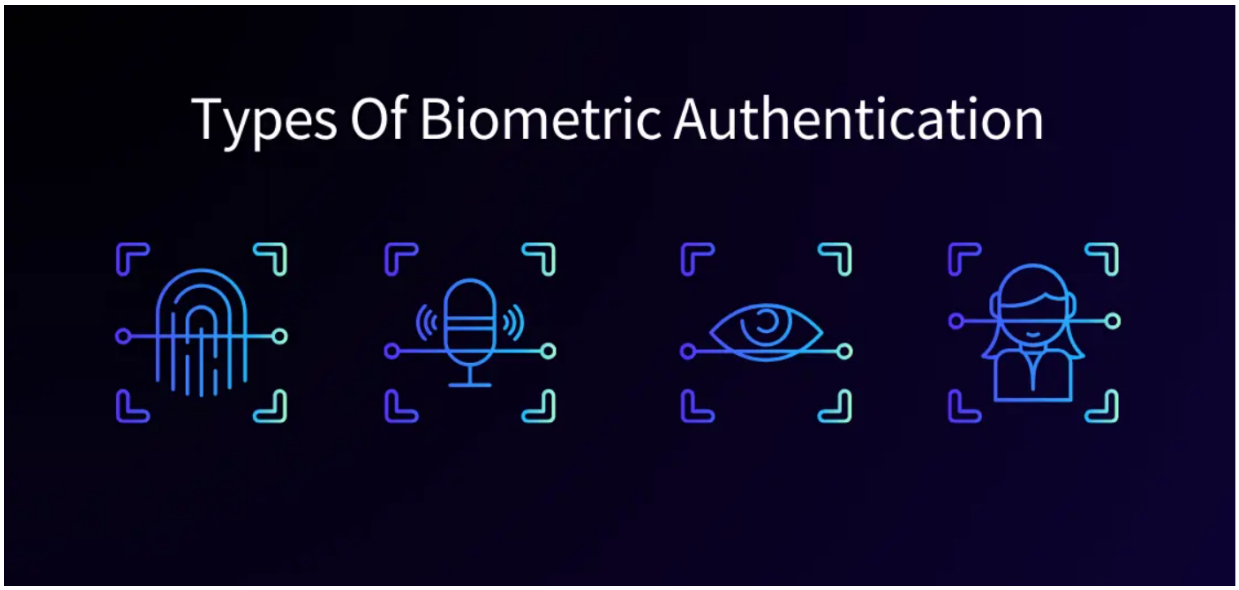


3. **Token-Based Authentication**:
   This method involves granting access based on a digital or physical token possessed by the user. Tokens can come in the form of hardware devices (like key fobs) or software tokens passed to a browser during login. Token-based systems are commonly used in enterprise environments to enhance security.

4. **Single Sign-On (SSO)**:
   SSO simplifies the login process by allowing users to access multiple applications through a single set of credentials. For instance, logging into Google or Facebook can grant access to various third-party apps linked to those accounts. SSO improves user experience while maintaining centralized control over authentication.

5. **Biometric Authentication**:
   Biometrics rely on unique biological traits, such as fingerprints, facial recognition, or voice patterns, to identify users. As biometric data is difficult to replicate, this method offers a high

level of security. Modern smartphones and laptops increasingly use fingerprint scanners or facial recognition to authenticate users.



6. **Multi-Factor Authentication (MFA)**:
   MFA combines two or more authentication methods to strengthen security. For example, a user might enter a password (something they know) and then verify their identity using a fingerprint scan (something they are). By requiring multiple factors, MFA significantly reduces the risk of unauthorized access.

Authorization: Defining Access Rights

While authentication confirms a user's identity, authorization determines what actions or resources they are allowed to access once authenticated. In other words, authorization answers the question: "What can this user do?" For example, after a file server verifies a user's identity through

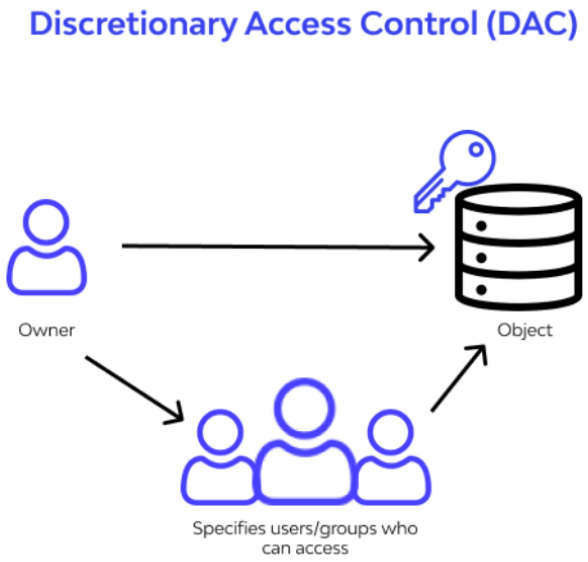
authentication, it checks their permissions to decide whether they can read, write, or delete specific files.

Authorization is crucial for enforcing organizational policies and protecting sensitive data. Without proper controls, users could inadvertently or intentionally access confidential information or perform unauthorized operations. Below, we will explore some common authorization models.

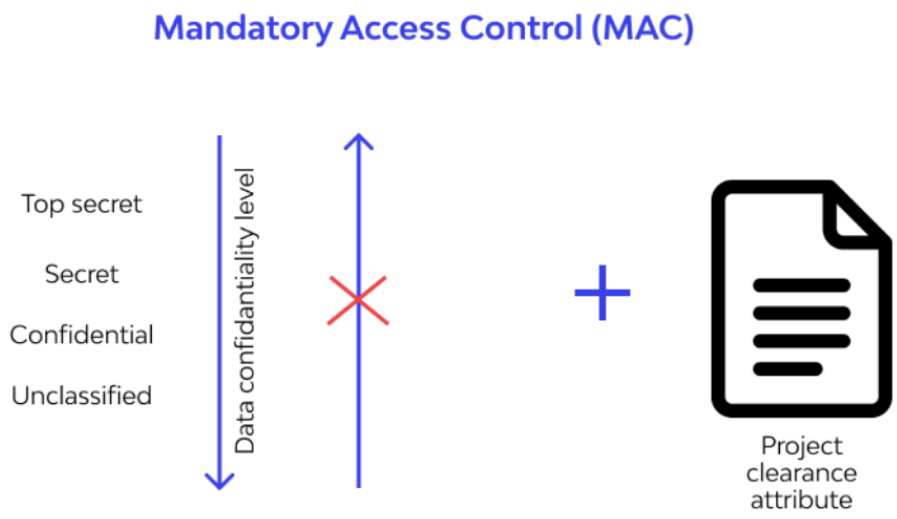
Types of Authorization Models

Different authorization models cater to varying needs and security requirements. Here are some of the most widely used approaches:

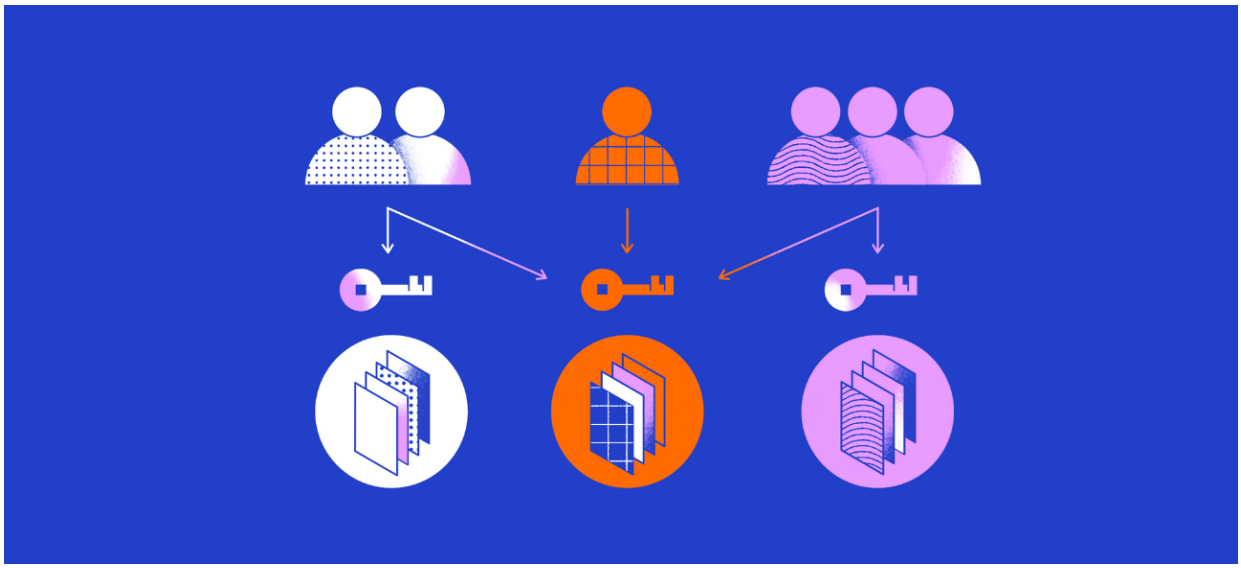
1. **Discretionary Access Control (DAC)**:
   DAC allows individual users or administrators to define access permissions for specific resources. For instance, a document owner might grant read-only access to colleagues while reserving editing rights for themselves. While flexible, DAC can sometimes lead to inconsistent or overly permissive settings if not carefully managed.



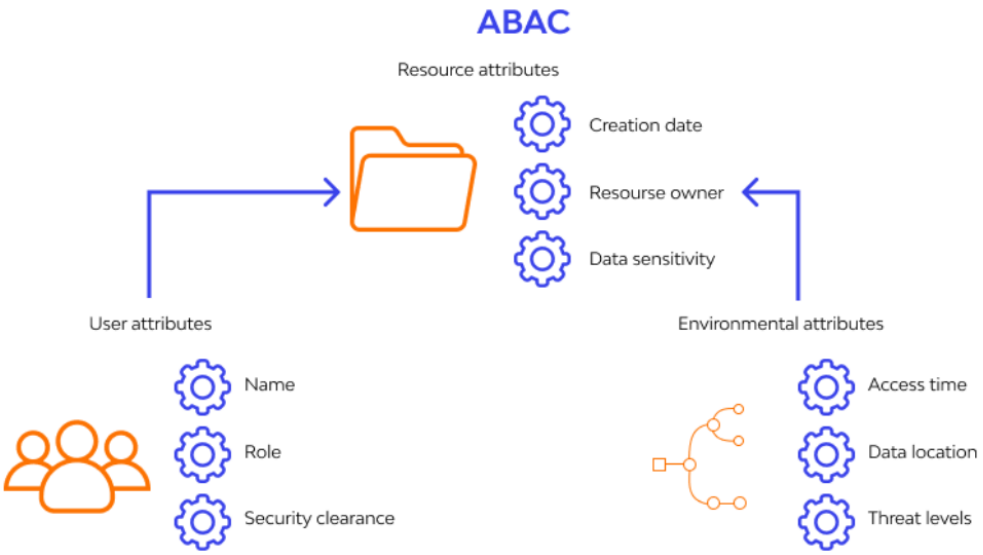
2. **Mandatory Access Control (MAC)**:
   Access rights are controlled by an essential force, which is subject to various levels of safety. The required permission control incorporates distributing depictions to structure assets as well as the security component or working framework. Only clients or devices with the basic data exceptional status have access to ensured assets. Relationships with varying degrees of information depiction, such as government and military affiliations, commonly use MAC to engineer all end clients. To perform MAC, you can use work-based acceptance control.



3. **Role-Based Access Control (RBAC)**:
   RBAC assigns permissions based on predefined roles within an organization. For example, an "admin" role might have full access to all system functions, while a "guest" role may only view public content. By tying access rights to roles rather than individual users, RBAC simplifies management and reduces the risk of errors.



4. **Attribute-Based Access Control (ABAC)**:
   ABAC takes a policy-driven approach, using attributes to determine access rights. Attributes can include details about the user (e.g., department, job title), the resource (e.g., file type, sensitivity level), or the environment (e.g., time of day, location). The system evaluates these attributes against predefined policies to decide whether access should be granted. ABAC provides granular control, making it suitable for complex scenarios.



Authentication Strategies

There are several authentication strategies available, each with its own advantages and disadvantages, and choosing the right strategy depends on the specific requirements of the system and security concerns.

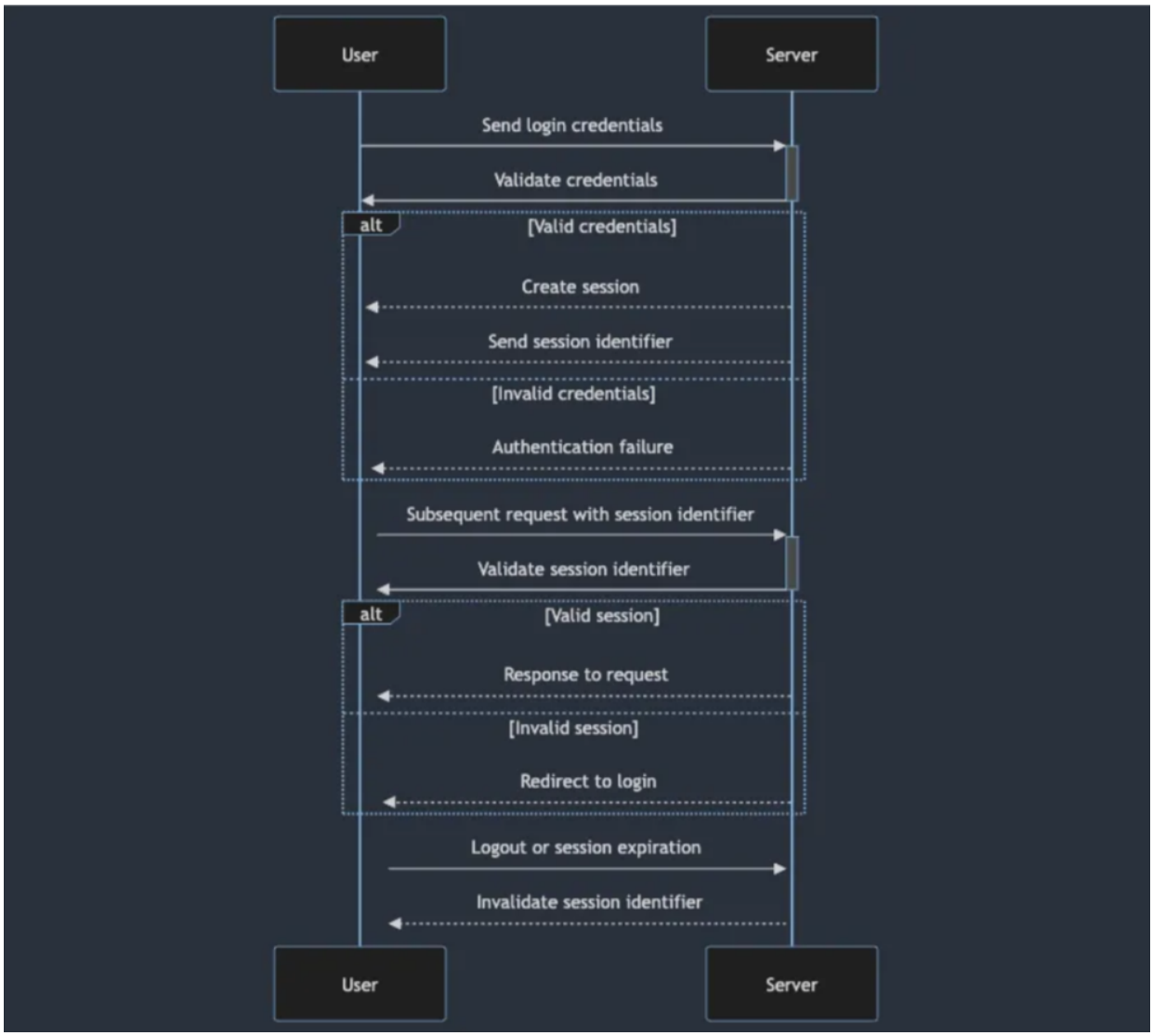
In some cases, a combination of several strategies is used to provide an additional layer of security and meet the diverse needs of users.

1. Session-Based Authentication

Session-based authentication began to be widely adopted on the web as web applications became more dynamic and interactive. Initially, sessions were primarily implemented using cookies, which stored a unique session identifier in the user's browser. The server then associated this identifier with the user's authentication data stored in its memory. Over time, security practices evolved, and new methods for managing sessions were introduced. This included implementing more secure session tokens and techniques to guard against cross-site request forgery (CSRF) attacks.

Workflow

The workflow involves the following steps:



1. **User Authentication**: The user provides login credentials (e.g., username and password).
2. **Credential Validation**: The server validates the credentials against a secure storage system (e.g., a database).
3. **Session Creation**: Upon successful validation, the server creates a unique session and assigns a session identifier to it.
4. **Session Storage**: The session ID is sent to the client's browser via a cookie. Additional session data may be stored on the server.
5. **Subsequent Requests**: For future requests, the browser sends the session ID back to the server, allowing it to retrieve the associated session data and authenticate the user.
6. **Session Termination**: The session is invalidated when the user logs out or after a predefined timeout period.

Advantages	Limitations
Provides a seamless user experience by maintaining session state across requests.	Requires server-side storage for session data, which can increase resource usage.
Supports advanced security measures like CSRF protection.	Vulnerable to session hijacking if cookies are not transmitted over HTTPS.

Use Cases

- Traditional web applications requiring personalized user experiences.

- E-commerce platforms tracking shopping carts and user activity.
- Content Management Systems (CMS) managing access to content and administrative functionalities.
- Enterprise applications controlling employee access to sensitive resources.

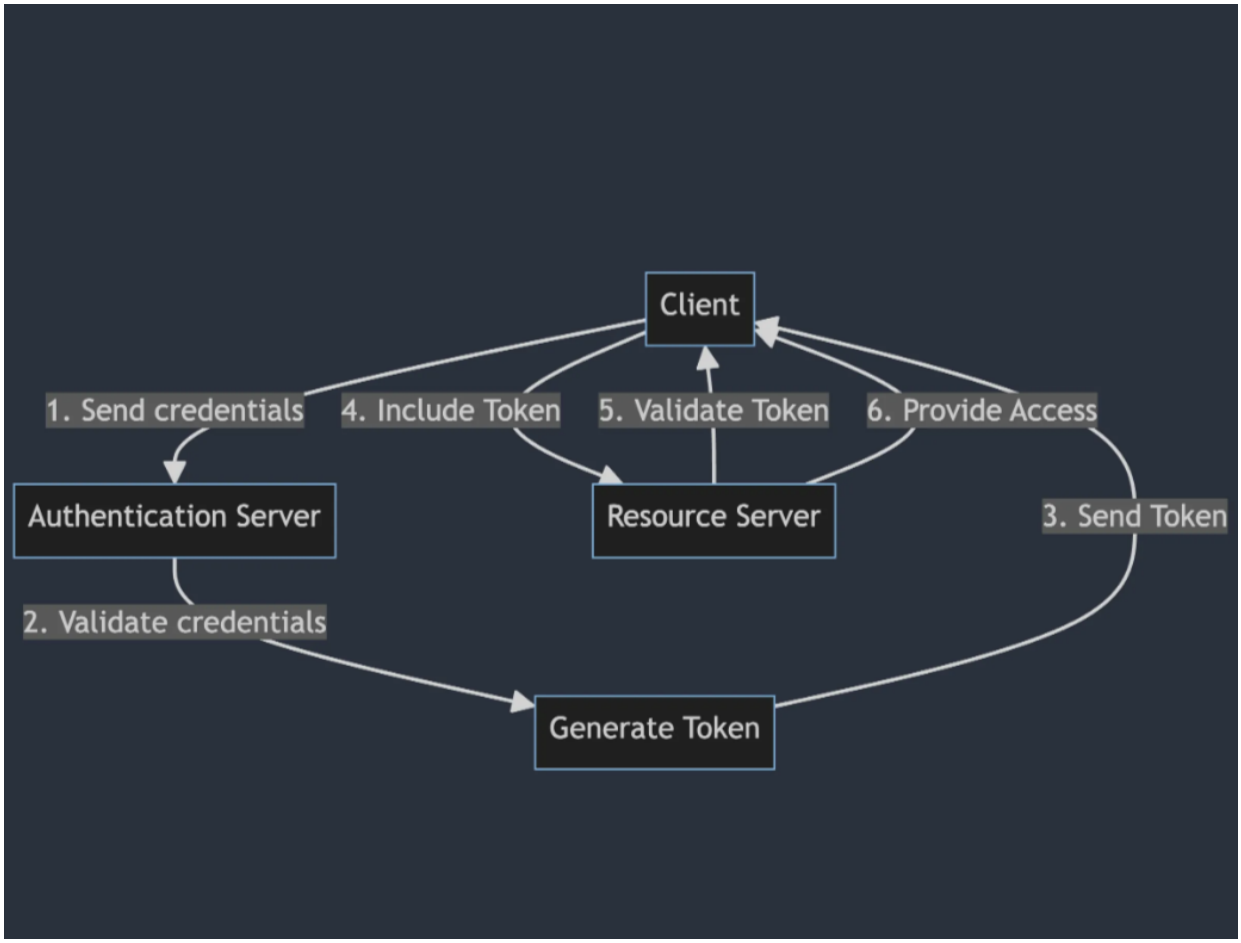
---

2. Token-Based Authentication

Token-based authentication emerged as a solution to address the limitations of traditional session-based methods, particularly in distributed and scalable environments. Before its adoption, systems relied heavily on server-side session management, which proved inefficient for modern architectures like Service-Oriented Architectures (SOA) and RESTful APIs. In the 2000s, with the rise of these architectures, token-based authentication gained popularity due to its stateless nature and flexibility.

Methodology

The workflow involves the following steps:



In Token Based Authentication, when a user provides their authentication credentials (such as username and password) to access a system, the authentication server validates these credentials and, if correct, generates an access token. This token is then sent back to the client, which includes it in each subsequent request to access protected resources.

Tokens can be of different types, such as JSON Web Tokens (JWT), OAuth tokens, among others. They are usually encrypted and contain information about the authenticated user and other

relevant authorization information.

When the server receives a request containing a token, it verifies the token's validity, usually through a digital signature or another integrity verification mechanism. If the token is valid and authorized to access the requested resource, the server processes the request.

Advantages	Limitations
Statelessness: Eliminates the need for server-side session storage, making it highly scalable and ideal for distributed systems.	Tokens can become large if excessive data is embedded in their payload.
Portability: Tokens can be used across multiple domains and services, enhancing interoperability.	Revoking tokens requires implementing additional mechanisms, such as blacklisting or short-lived tokens with refresh strategies.
Security: Encrypted and signed tokens prevent tampering and unauthorized access.	Vulnerable to token theft if transmitted over unsecured connections (mitigated by using HTTPS).
Expiration: Tokens can have a defined lifespan, reducing the risk of misuse if compromised.	

Use Cases

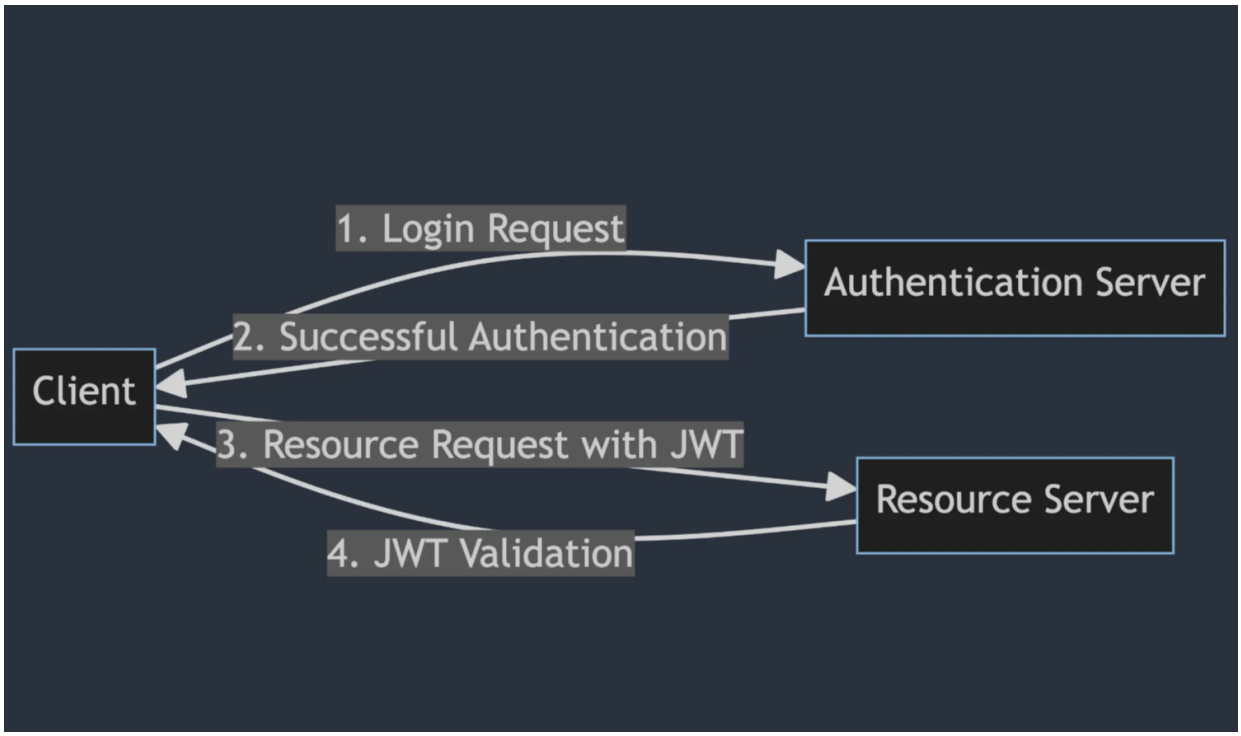
- **Web Applications and RESTful APIs**: Token-based authentication is widely used in stateless interactions, where traditional sessions are impractical.
- **Single Sign-On (SSO)**: In corporate environments, tokens enable users to access multiple applications with a single login, streamlining authentication workflows.
- **Mobile Applications**: Mobile apps benefit from token-based authentication due to its scalability and efficient handling of requests.
- **Microservices and Service-Oriented Architectures**: In distributed systems, tokens simplify secure communication between services by centralizing authentication and authorization logic.

---

3. JWT (JSON Web Tokens)

Digital security is an increasingly pressing concern as technology advances. Among the many tools and techniques employed to ensure system integrity, authentication via JSON Web Tokens (JWT) emerges as a powerful and flexible strategy. In this article, we explore the history, functioning, and use cases of this authentication approach. JWT originated in the context of the modern web and arose from the need for a secure and efficient way to transmit authentication information between parties. It was first introduced in 2010 as an open standard (RFC 7519) by the IETF (Internet Engineering Task Force) and has since gained popularity in many applications requiring authentication and authorization.

Methodology



JWT is a compact, self-contained format for securely transmitting information between two parties.

It consists of three parts:

1. **Header**: Contains metadata about the token type and the encryption algorithm used to sign the token.
2. **Payload**: This is where the token data is stored. It can include information such as user identity and permissions.
3. **Signature**: It is the final part of the token, used to verify if the token has been tampered with during transmission. The signature is generated by combining the header, payload, and a secret key using a hashing

Upon receiving a JWT, the recipient can verify the signature to ensure the authenticity of the token and then access the data contained in the payload.

The server verifies the token's signature and extracts the payload to authenticate the user.

Advantages	Limitations
Stateless: No need for server-side session storage, reducing resource overhead.	Tokens can become large if excessive data is embedded in the payload.
Portable: Can be used across different domains and services.	Revoking tokens requires implementing additional mechanisms, such as blacklisting.
Secure: Encrypted and signed tokens prevent tampering.	

Use Cases

- User authentication in web and mobile applications.
- Authorization and access control by embedding user roles and permissions in the token.

- Microservices architectures where services need to authenticate and authorize requests without relying on a central session store.
- Single Sign-On (SSO) solutions enabling seamless access to multiple applications.

Passport.js

Passport.js is an authentication middleware designed specifically for Node.js applications. It provides a modular approach to implementing authentication strategies, allowing developers to easily integrate methods like:

- **Local Authentication**: Verifying users with a username and password.
- **OAuth**: Allowing users to log in via third-party services like Google, Facebook, or GitHub.
- **OpenID**: Supporting decentralized identity verification.

Passport.js integrates seamlessly with Express.js, enabling developers to define routes and enforce authentication at specific endpoints. By using session management, Passport.js ensures that authenticated users remain logged in across multiple requests.

Steps to Use Passport.js Middleware in an Express Application

Step 1: Create a New Project Folder

Start by creating a new folder for your project and navigating into it using the terminal:

```
mkdir passport-auth-example
cd passport-auth-example
```

Step 2: Initialize the Project

Initialize the project with `npm` to generate a `package.json` file:

```
npm init -y
```

Step 3: Install Required Dependencies

Install the necessary dependencies for your project:

```
npm install express passport passport-local express-session
```

These packages include:

- **express**: The core framework for building the server.
- **passport**: The authentication middleware.
- **passport-local**: A strategy for local username/password authentication.
- **express-session**: Middleware for managing user sessions.

Step 4: Project Structure

After installing the dependencies, your `package.json` file should look like this:

```
"dependencies": {
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "passport": "^0.7.0",
    "passport-local": "^1.0.0"
}
```

Role of Passport.js in Express Application Authentication

Passport.js plays a critical role in securing an Express application by implementing authentication strategies. Below is an example of how Passport.js handles local authentication using a username and password:

1. **Local Strategy Configuration**:
   Passport.js uses the `LocalStrategy` to authenticate users based on their credentials. It serializes and deserializes user data to manage session storage.
2. **Route Definitions**:
   Routes such as `/login`, `/profile`, and `/logout` are defined to handle user interactions. The `/profile` route is protected, ensuring only authenticated users can access it.
3. **Session Management**:
   The `express-session` middleware tracks authenticated users by storing session data. This allows users to remain logged in across multiple requests.

Example of the Implementation

Below is a complete example of integrating Passport.js into an Express application. Save this code in a file named `app.js`.

```
// app.js
const express = require('express');
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const session = require('express-session');
```

```
const app = express();

// Configure Passport.js Middleware
passport.use(new LocalStrategy(
  (username, password, done) => {
    // Simulate a user database with demo credentials
    if (username === 'admin' && password === 'gfg') {
      return done(null, { id: 1, username: 'user' });
    } else {
      return done(null, false, { message: 'Incorrect username or password.'
});
    }
  }
));

// Serialize and Deserialize User for Session Management
passport.serializeUser((user, done) => {
  done(null, user.id);
});

passport.deserializeUser((id, done) => {
  // Simulate retrieving user data from a database
  const user = { id: 1, username: 'user' };
  done(null, user);
});

// Middleware Setup
app.use(express.urlencoded({ extended: true }));
app.use(session({
  secret: 'gfg', // Secret key for session encryption
  resave: false,
  saveUninitialized: false
}));
app.use(passport.initialize());
app.use(passport.session());

// Define Routes
app.get('/', (req, res) => {
  res.send('<h1>Passport.js Authentication Example</h1>');
});

app.get('/login', (req, res) => {
  res.send(`
    <h1>Login Page</h1>
    <form action="/login" method="post">
      Username: <input type="text" name="username"><br>
      Password: <input type="password" name="password"><br>
      <input type="submit" value="Login">
    </form>
  `);
});

app.post('/login',
  passport.authenticate('local', {
    successRedirect: '/profile'
```

```
    successRedirect: '/profile',
    failureRedirect: '/login',
    failureFlash: true
  })
);

app.get('/profile', isAuthenticated, (req, res) => {
  res.send(`
    <h1>Welcome ${req.user.username}!</h1>
    <a href="/logout">Logout</a>
  `);
});

app.get('/logout', (req, res) => {
  req.logout((err) => {
    if (err) return next(err);
    res.redirect('/');
  });
});

// Middleware to Check Authentication
function isAuthenticated(req, res, next) {
  if (req.isAuthenticated()) {
    return next();
  }
  res.redirect('/login');
}

// Start the Server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Setting Up Secure Login Endpoints and Token-Based Authentication

Token-based authentication is a secure approach to managing user sessions in web applications.

Create a Login Endpoint

- The login endpoint accepts user credentials (e.g., username and password) and validates them against a database.
- If the credentials are valid, the server generates a token (e.g., a JWT) and sends it back to the client.

Example Code:

```
const express = require('express');
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const app = express();

// Middleware to parse JSON
app.use(express.json());

// Mock user database
const users = [
  { id: 1, username: 'admin', password: '$2b$10$hashedpassword' } //
Replace with hashed password
];

// Login endpoint
app.post('/login', async (req, res) => {
  const { username, password } = req.body;

  // Find the user in the database
  const user = users.find(u => u.username === username);
  if (!user) return res.status(401).send('Invalid username or password');

  // Compare the provided password with the hashed password
  const isValidPassword = await bcrypt.compare(password, user.password);
  if (!isValidPassword) return res.status(401).send('Invalid username or
password');

  // Generate a JWT token
  const token = jwt.sign({ userId: user.id }, 'your-secret-key', {
expiresIn: '1h' });

  // Send the token to the client
  res.json({ token });
});
```

Protect API Endpoints with Token Verification

- Once the client receives the token, it includes it in the `Authorization` header of subsequent requests (e.g., `Bearer <token>`).
- The server verifies the token before granting access to protected resources.

Example Middleware for Token Verification:

```
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Extract token
from "Bearer <token>"
  if (!token) return res.sendStatus(401); // Unauthorized

  jwt.verify(token, 'your-secret-key', (err, user) => {
    if (err) return res.sendStatus(403); // Forbidden
    req.user = user; // Attach the decoded user to the request object
    next();
  });
}

// Protected endpoint
app.get('/protected', authenticateToken, (req, res) => {
  res.send(`Welcome, User ID: ${req.user.userId}`);
});
```

Implementing Role-Based Access Control (RBAC) to Protect API Endpoints

Role-Based Access Control (RBAC) is a widely used method for enforcing access restrictions based on predefined roles. Each role has specific permissions, and users are assigned one or more roles. RBAC ensures that only authorized users can access certain resources or perform specific actions.

Steps to Implement RBAC in an Express Application

Define Roles and Permissions

Create a mapping of roles to their respective permissions.

Example:

```
const roles = {
  admin: ['read', 'write', 'delete'],
  editor: ['read', 'write'],
  viewer: ['read']
};
```

Assign Roles to Users

Store user roles in your database or mock data.

Example:

```
const users = [
  { id: 1, username: 'admin', password: '$2b$10$hashedpassword', role:
'admin' },
  { id: 2, username: 'editor', password: '$2b$10$hashedpassword', role:
'editor' }
];
```

Create a Middleware to Check Permissions

Verify that the authenticated user's role has the required permission for accessing a specific endpoint.

Example Middleware:

```
function authorize(permission) {
  return (req, res, next) => {
    const userRole = req.user.role; // Extract role from the authenticated
user
    if (!roles[userRole] || !roles[userRole].includes(permission)) {
      return res.status(403).send('Forbidden: Insufficient permissions');
    }
    next();
  };
}
```

Protect Endpoints Based on Roles

Apply the `authorize` middleware to enforce role-based access control.

Example:

```
// Admin-only endpoint
app.get('/admin-dashboard', authenticateToken, authorize('read'), (req,
res) => {
  res.send('Welcome to the Admin Dashboard');
});

// Editor-only endpoint
app.post('/create-post', authenticateToken, authorize('write'), (req, res)
=> {
  res.send('Post created successfully');
});

// Viewer-only endpoint
app.get('/view-posts', authenticateToken, authorize('read'), (req, res) =>
{
  res.send('Here are the posts');
});
```

Conclusion

Access control is essential for safeguarding systems and ensuring that only authorized users can access protected resources. Through authentication, we verify the identity of users, while authorization defines the level of access they are granted. By leveraging modern authentication methods, such as MFA and token-based systems, and implementing robust authorization models like RBAC, organizations can enhance security and streamline user management. Tools like Passport.js and frameworks like Express.js provide practical ways to integrate these concepts into real-world applications. Understanding and applying these principles will empower you to build secure, scalable, and user-friendly systems.